

Lab Practicum 01: Empirical Parallel Computing

Silicon Profiling, Multi-Core Scaling Limits, and Concurrency Hazards

STUDENT NAME Kamilla	STUDENT ID 230103129	HOST CPU / SOC MODEL	OPERATING SYSTEM & KERNEL
-------------------------	-------------------------	----------------------	---------------------------

Generative AI & Tooling Policy: You are explicitly authorized to use LLMs (ChatGPT, Claude, GitHub Copilot) to generate code scripts in any language (C, C++, Python, Go, Rust, Java). However: All execution timings, memory counters, and hardware cache metrics must be physically benchmarked on your host machine. Fabricated numbers receive zero credit.

TASK 1: Host Silicon Audit & Flynn’s Taxonomy Mapping 15 Minutes | 20 Points

Query your operating system and CPU registers to document physical core topology, cache boundaries, and vector math extensions.

Terminal Audit Commands: Linux: lscpu && lscpu --caches macOS: sysctl -a grep machdep.cpu Windows: Run wmic cpu get /format:list and inspect Task Manager > Performance > CPU.			
Hardware Property	Local Measured Value	Hardware Property	Local Measured Value
CPU Model & Architecture	Intel Pentium Silver N5000, x86-64	Base & Max Boost Clock	1.10 GHz 1.101 GHz*
Physical Hardware Cores	4	Logical Cores (SMT/Threads)	4
L1 Data / Instruction Cache	224KB	L2 Cache (Per Core / Total)	4MB
L3 Shared Last-Level Cache	Not available	SIMD Vector Extensions	[AVX-2, AVX-512, ARM NEON]

```
C:\Users\leap>wmic cpu get Name,NumberOfCores,NumberOfLogicalProcessors,MaxClockSpeed
MaxClockSpeed Name                                NumberOfCores  NumberOfLogicalProcessors
1101           Intel(R) Pentium(R) Silver N5000 CPU @ 1.10GHz 4              4
```

Q1.1: Map your host machine's execution modes into Flynn's Taxonomy:

SISD (Single Instruction, Single Data): A single physical CPU core executing a scalar, single-threaded program (e.g., a simple sequential C/Python script or single-threaded compiler pass) operating on one data element at a time.

SIMD (Single Instruction, Multiple Data): Vector hardware extensions (such as AVX2, AVX-512, or ARM NEON) executing a single vector instruction simultaneously across wide registers containing multiple data elements (e.g., vector/matrix arithmetic operations).

MIMD (Multiple Instruction, Multiple Data): Multiple independent CPU cores/threads running asynchronously, executing different instruction streams on separate dataset chunks simultaneously (e.g., multi-core OS thread scheduling, OpenMP tasks, or multiprocessing workloads).

TASK 2: Multi-Thread Scaling Benchmark & Contention Wall 35 Minutes | 30 Points

Write or prompt an AI to generate a compute-bound CPU benchmark parameterized by thread count N. Run 3 iterations per thread count and log wall-clock execution time.

Workload Algorithm: [e.g., Prime Sieve to 5,000,000 / Matrix Multiply]				Language & Runtime: [e.g., C OpenMP / Python multiprocessing / Go]		
Threads (N)	Run 1 (s)	Run 2 (s)	Run 3 (s)	Avg Time TN (s)	Speedup SN = T1/TN	Efficiency EN = SN/N
N = 1 (Baseline)	18.534	30.462	26.195	25.064	1.00x	100.0%
N = 2	14.266	11.377	12.800	12.814	1.96x	97.8%
N = 4	6.254	9.676	9.367	8.432	2.97x	74.3 %
N = 8	9.902	5.566	5.000	6.823	3.67 x	45.9 %
N = 16	8.336	6.999	14.435	9.923	2.53x	15.8 %
N = 32	19.337	12.692	10.568	14.199	1.77 x	5.5%

Language & Runtime: Python multiprocessing							
Threads (N)	Run 1 (s)	Run 2 (s)	Run 3 (s)	Avg (s)	Speedup	Efficiency	
N=1	18.534	30.462	26.195	25.064	1.00	x	100.0 %
N=2	14.266	11.377	12.800	12.814	1.96	x	97.8 %
N=4	6.254	9.676	9.367	8.432	2.97	x	74.3 %
N=8	9.902	5.566	5.000	6.823	3.67	x	45.9 %
N=16	8.336	6.999	14.435	9.923	2.53	x	15.8 %
N=32	19.337	12.692	10.568	14.199	1.77	x	5.5 %
PS C:\Users\leap\AppData\Local\Programs\Microsoft VS Code>							
PS C:\Users\leap\AppData\Local\Programs\Microsoft VS Code>							

Q2.1: Scaling Saturation & Hardware Contention Analysis:

The speedup saturates (or declines) once the thread count exceeds **N =32** which corresponds to the host machine's physical core count.

- **Physical vs. Logical Cores:** Beyond the physical core limit, logical threads (SMT/Hyper-Threading) share execution resources (ALUs, L1/L2 caches) within the same core, yielding diminishing returns.
- **Overhead & Contention:** When $N > \text{Logical Cores}$ (e.g., $N=32$), excessive context switching overhead, OS thread scheduling delays, cache thrashing, and memory bandwidth contention severely reduce efficiency ($E_N \ll 100\%$).

TASK 3: The Unsynchronized Shared Counter & Race Condition Trap25 Minutes | 25 Points

Spawn 10 parallel threads. Each thread executes a loop incrementing a shared global integer 1,000,000 times without atomic operations or locks. Theoretical expected total: $10 \times 1,000,000 = 10,000,000$. Execute 10 consecutive runs.

Run	Measured Output	Error (10^7 - Act)	Run	Measured Output	Error (10^7 - Act)
#1	10000000	0	#6	10000000	0
#2	10000000	0	#7	10000000	0
#3	10000000	0	#8	10000000	0
#4	10000000	0	#9	10000000	0
#5	10000000	0	#10	10000000	0

Q3.1: Read-Modify-Write (RMW) Window

Value corruption occurs because `counter += 1` is not an atomic operation at the CPU machine instruction level. It consists of three distinct assembly steps:

- Load:** Read value from shared memory/L3 cache into a local CPU register.
- Add:** Increment the value inside the ALU register by 1.
- Store:** Write the updated register value back to shared memory.

When multiple threads execute concurrently without synchronization, their execution interleaves within this Read-Modify-Write window. For example, Thread A and Thread B may both **Load** value `100` simultaneously, both **Add** 1 to get `101`, and both **Store** `101`. As a result, two increment operations occurred, but the global counter only increased by 1, causing data loss (Race Condition).

Q3.2: The Synchronization Penalty

Race-free synchronized code is substantially slower because acquiring and releasing Mutexes/Locks introduces severe performance overheads:

- Thread Serialization:** Mutexes force execution to become sequential for critical sections, destroying parallel speedup.
- OS Context Switching & Blocking:** Unacquired locks cause threads to block/sleep, triggering expensive OS kernel context switches.
- Cache Coherency & Bus Locking:** Atomic lock instructions force CPU cores to invalidate shared lines across L1/L2 caches via bus locking protocols (e.g., MESI protocol), stalling the pipeline.

TASK 4: Empirical Amdahl Verification vs. Gustafson Scaling Horizon15 Minutes | 15 Points

Use your empirical single-thread time (T1) and two-thread time (T2) from Task 2 to derive the parallel fraction p of your workload, then evaluate theoretical performance bounds.

1. Parallel Fraction Derivation (p) Formula: $p = 2 \times (T1 - T2) / T1$ T1 = 25,064____s T2 =12,814____s Calculated p = 0,977_____(decimal between 0.0 and 1.0)	2. Amdahl Theoretical Speedup Ceiling Formula: $S_{max} = 1 / (1 - p)$ Sequential fraction $(1 - p) = 0,022$ _____ Max Theoretical Speedup ($N \rightarrow \infty$) = 44,43____x
3. Amdahl 64-Core Projection Formula: $S_{64} = 1 / [(1 - p) + (p / 64)]$ Projected Speedup on 64 Cores = 26,47____x	4. Gustafson Scaled Speedup (Weak Scaling) Formula: $S_{Gustafson} = (1 - p) + (p \times 64)$ Projected Scaled Speedup (64x Problem) = 62,58____x

Q4.1: Strong vs. Weak Scaling Analysis:

Amdahl's Law (Strong Scaling): Assumes a **fixed problem size**. As core count N **goes to infinity**, the execution time of the parallel portion shrinks toward zero, making the serial fraction $(1 - p)$ the absolute bottleneck that caps maximum speedup regardless of how many cores are added.

Gustafson's Law (Weak Scaling): Assumes that as hardware resources scale, the **problem size expands proportionally** to utilize the available hardware. Since the workload scales up with core count, the parallel execution time dominates the fixed sequential overhead, enabling near-linear performance scaling on highly parallel nodes (such as 64-core systems).